

Tugas 2: Machine Learning – Artificial Neural Network (ANN)

Muhammad Zaidan Ramdhan - 0110222040

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: muha22040ti@student.nurulfikri.ac.id

Laporan ini bertujuan untuk menerapkan Artificial Neural Network (ANN) dengan arsitektur Multi-Layer Perceptron (MLP) pada dataset citra MNIST (Handwritten Digits). Dataset MNIST terdiri dari gambar angka tulisan tangan berukuran 28×28 piksel dengan sepuluh kelas (0–9), sehingga permasalahan yang dihadapi adalah klasifikasi multi-kelas. Melalui percobaan ini, mahasiswa diharapkan dapat memahami penerapan ANN pada data citra serta mengamati performa model MLP dalam mengklasifikasikan data non-tabular.

Pada tahap preprocessing, setiap citra diubah menjadi vektor satu dimensi melalui proses flattening dan dinormalisasi ke dalam rentang 0–1 untuk meningkatkan stabilitas pelatihan model. Label kelas dikonversi ke dalam format one-hot encoding, kemudian model MLP dilatih menggunakan beberapa lapisan tersembunyi dengan fungsi aktivasi ReLU dan lapisan output Softmax. Hasil percobaan menunjukkan bahwa model mampu mencapai akurasi yang tinggi pada data pengujian, sehingga dapat disimpulkan bahwa MLP efektif digunakan untuk klasifikasi citra sederhana dan memberikan perbedaan proses yang jelas dibandingkan penerapan ANN pada dataset tabular.

1. Tugas mandiri – Latihan

```
!pip install kaggle

Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: certifi<=14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2025.11.12)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.4)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.5)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)

kaggle = '/content/kaggle.json'
```

Gambar 1. 2

Pada *Gambar* di atas, merupakan sebuah code untuk menginstall kaggle

```
kaggle = '/content/kaggle.json'
```

Gambar 1. 3

Membuat path untuk kaggle

```
!kaggle datasets download -d olafkrastovski/handwritten-digits-0-9

Dataset URL: https://www.kaggle.com/datasets/olafkrastovski/handwritten-digits-0-9
License(s): CC0-1.0
handwritten-digits-0-9.zip: Skipping, found more recently modified local copy (use --force to force download)
```

Gambar 1. 4

Mendownload dataset yang berasal dari kaggle

```

from zipfile import ZipFile
import os
file_name = 'handwritten-digits-0-9.zip'

extract_folder = "dataset"
os.makedirs(extract_folder, exist_ok=True)

with ZipFile(file_name, 'r') as zip:
    zip.extractall(extract_folder)
    print('Done')

```

Done

Gambar 1. 5

Mengestrak file dataset dengan file zip

```

data_0 = os.listdir('/content/dataset/0')
data_1 = os.listdir('/content/dataset/1')
data_2 = os.listdir('/content/dataset/2')
data_3 = os.listdir('/content/dataset/3')
data_4 = os.listdir('/content/dataset/4')
data_5 = os.listdir('/content/dataset/5')
data_6 = os.listdir('/content/dataset/6')
data_7 = os.listdir('/content/dataset/7')
data_8 = os.listdir('/content/dataset/8')
data_9 = os.listdir('/content/dataset/9')

```

Gambar 1. 6

Menginisiasi data gambar dari tiap dataset

```

import numpy as np
import os
import cv2
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

```

Gambar 1.7

Melakukan import library yang akan kita gunakan

```
dataset_path = "/content/dataset"

X = []
y = []

for label in range(10):
    folder_path = os.path.join(dataset_path, str(label))
    for file in os.listdir(folder_path):
        img_path = os.path.join(folder_path, file)

        img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

        # resize agar konsisten (28x28)
        img = cv2.resize(img, (28, 28))

        X.append(img)
        y.append(label)
```

Gambar 1.8

Dataset disusun dalam folder berdasarkan kelas digit (0–9). Setiap gambar dibaca dalam format grayscale dan diubah ukurannya menjadi 28×28 piksel agar konsisten. Data citra disimpan ke variabel X, sedangkan label kelas disimpan ke y

```
X = np.array(X)
y = np.array(y)

print("Shape X:", X.shape)
print("Shape y:", y.shape)

Shape X: (21555, 28, 28)
Shape y: (21555,)
```

Gambar 1.9

Data dikonversi ke format NumPy array agar dapat diproses oleh model neural network. Aktivasi Softmax digunakan untuk menghasilkan probabilitas tiap kelas.

```
X = X.reshape(X.shape[0], 28 * 28)
```

Gambar 1.10

Citra dua dimensi (28×28) diubah menjadi vektor satu dimensi sepanjang 784 fitur karena MLP hanya menerima input vektor.

```
X = X / 255.0
```

Gambar 1.11

Nilai piksel dinormalisasi ke rentang 0–1 untuk meningkatkan stabilitas dan kecepatan pelatihan model.

```
y = to_categorical(y, 10)
```

Gambar 1.12

Label kelas dikonversi ke bentuk one-hot encoding karena permasalahan bersifat klasifikasi multi-kelas.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
print("X_train:", X_train.shape)  
print("X_test:", X_test.shape)  
  
X_train: (17244, 784)  
X_test: (4311, 784)
```

Gambar 1.13

Dataset dibagi menjadi 80% data training dan 20% data testing untuk mengevaluasi kemampuan generalisasi model.

```
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(784,)))
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_9 (Dense)	(None, 128)	100,480
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Gambar 1.14

Model MLP terdiri dari satu input layer, dua hidden layer dengan aktivasi ReLU, serta output layer dengan aktivasi Softmax untuk menghasilkan probabilitas setiap kelas digit.

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Gambar 1.15

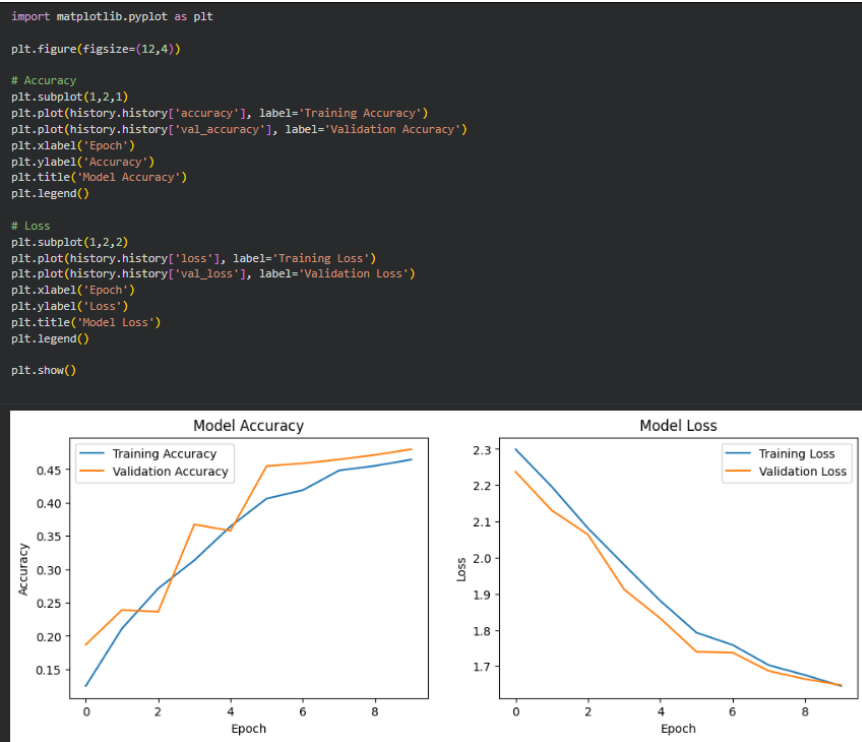
Adam optimizer digunakan karena efisien dan adaptif. Categorical crossentropy sesuai untuk klasifikasi multi-kelas.

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=10,  
    batch_size=128,  
    validation_split=0.2  
)
```

Epoch 1/10	100/100	4s 16ms/step	- accuracy: 0.1102	- loss: 2.3336	- val_accuracy: 0.1870	- val_loss: 2.2370
Epoch 2/10	100/100	1s 9ms/step	- accuracy: 0.1956	- loss: 2.2187	- val_accuracy: 0.2389	- val_loss: 2.1305
Epoch 3/10	100/100	1s 5ms/step	- accuracy: 0.2540	- loss: 2.1090	- val_accuracy: 0.2363	- val_loss: 2.0638
Epoch 4/10	100/100	1s 5ms/step	- accuracy: 0.2868	- loss: 2.0287	- val_accuracy: 0.3674	- val_loss: 1.9124
Epoch 5/10	100/100	1s 5ms/step	- accuracy: 0.3625	- loss: 1.9001	- val_accuracy: 0.3578	- val_loss: 1.8325
Epoch 6/10	100/100	1s 8ms/step	- accuracy: 0.3983	- loss: 1.8097	- val_accuracy: 0.4549	- val_loss: 1.7402
Epoch 7/10	100/100	1s 8ms/step	- accuracy: 0.4222	- loss: 1.7605	- val_accuracy: 0.4590	- val_loss: 1.7377
Epoch 8/10	100/100	1s 7ms/step	- accuracy: 0.4422	- loss: 1.7157	- val_accuracy: 0.4648	- val_loss: 1.6871
Epoch 9/10	100/100	1s 5ms/step	- accuracy: 0.4562	- loss: 1.6804	- val_accuracy: 0.4717	- val_loss: 1.6649
Epoch 10/10	100/100	1s 5ms/step	- accuracy: 0.4684	- loss: 1.6422	- val_accuracy: 0.4801	- val_loss: 1.6481

Gambar 1.16

Model dilatih selama 10 epoch dengan 20% data training digunakan sebagai data validasi untuk memantau performa selama pelatihan.



Gambar 1.17

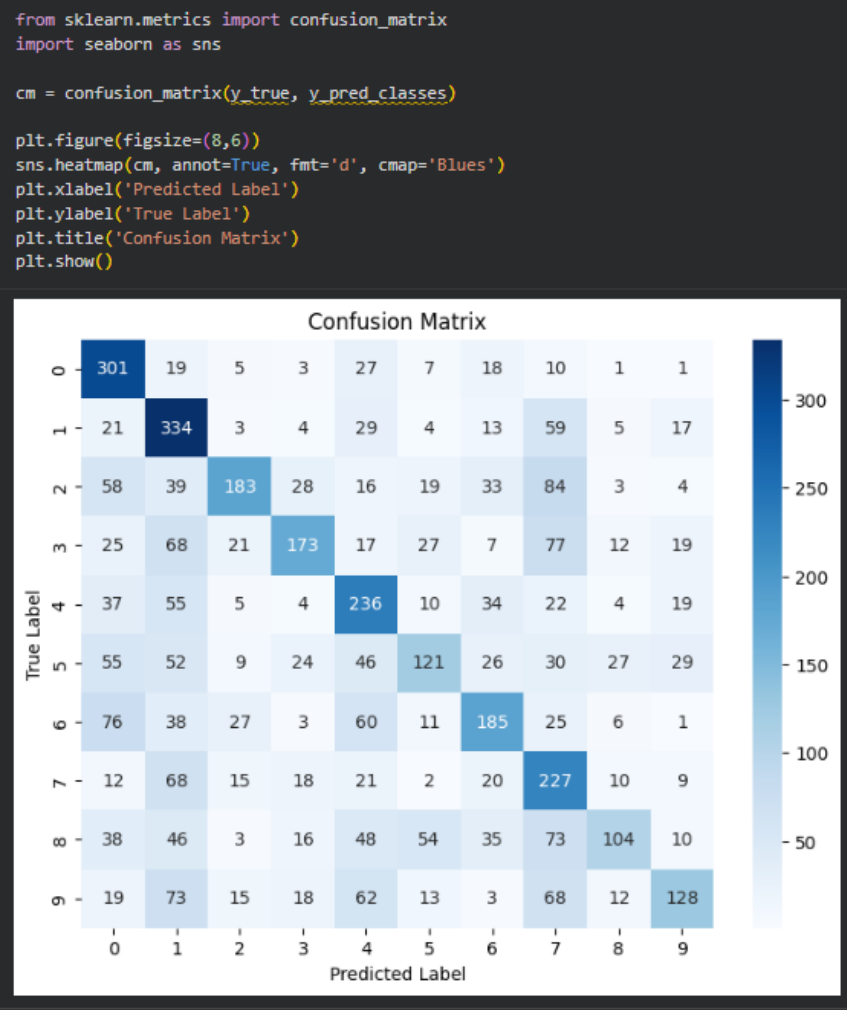
Grafik ini digunakan untuk melihat tren peningkatan akurasi dan penurunan loss selama proses pelatihan serta mendeteksi overfitting.

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_accuracy)
```

135/135 ————— 1s 4ms/step - accuracy: 0.4603 - loss: 1.6790
Test Accuracy: 0.46207377314567566

Gambar 1.18

Evaluasi dilakukan menggunakan data testing untuk mengukur performa model pada data yang belum pernah dilihat sebelumnya.



Gambar 1.19

Confusion matrix digunakan untuk melihat distribusi kesalahan dan keberhasilan klasifikasi pada setiap kelas digit.

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_true, y_pred_classes))
```

	precision	recall	f1-score	support
0	0.47	0.77	0.58	392
1	0.42	0.68	0.52	489
2	0.64	0.39	0.49	467
3	0.59	0.39	0.47	446
4	0.42	0.55	0.48	426
5	0.45	0.29	0.35	419
6	0.49	0.43	0.46	432
7	0.34	0.56	0.42	402
8	0.57	0.24	0.34	427
9	0.54	0.31	0.40	411
accuracy			0.46	4311
macro avg	0.49	0.46	0.45	4311
weighted avg	0.50	0.46	0.45	4311

Gambar 1.20

Classification report menampilkan nilai precision, recall, dan F1-score untuk setiap kelas

Link github praktikum:

https://github.com/MuhZaidanRamdhan/machine-learning/blob/main/pertemuan13/praktikum_13/notebooks/praktikum_13.ipynb

Link github tugas:

https://github.com/MuhZaidanRamdhan/machine-learning/blob/main/pertemuan13/praktikum_mandiri/notebooks/praktikum_mandiri.ipynb